Strings in Python

29 October 2026 05:56

What is a string in Python?

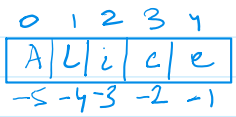
A string in Python is an immutable sequence of Unicode characters used to represent text data. Because strings are sequences, each character inside them occupies a specific position (index) starting from zero. Because they are immutable. Once a string is created, its character cannot be changed, modified, or overwritten in place.

Key Properties & Concepts

- **Creation:** Return by enclosing characters in single('…'), double("…"), or triple('''…''' or """…""")quotes. Triple codes are specifically used for multi-line strings and doc strings.
- **Immutability:** Modifying a character at a specific index directly (e.g., text[0] = 'A' raises a TypeError. Any operation that alters a string returns a completely new string object.

  Example:

```
>>> name = 'Bob'
>>> type(name)
<class 'str'>
>>> name[0]
'B'
>>> name[1]
'o'
>>> name[2]
'b'
>>> name[0] = 'D'
Traceback (most recent call last):
  File "<python-input-6>", line 1, in <module>
    name[0] = 'D'
    ~~~~^^^
TypeError: 'str' object does not support item assignment
>>> name = "Alice"
>>> name
'Alice'
>>> name[0]
'A'
```



Python says
first element 0th index
last element −1th index
Universally
last element N−1

- **Indexing:** Accessing individual characters using brackets [] Supports positive indexing (starting from zero from the left) And negative indexing. (Starting from minus one from the right.) For Example:

```
>>> name = "Alice"
>>> name
'Alice'
>>> name[0]
'A'
>>> name[-1]
'e'
>>> name[-2]
'c'
>>> name[len(name)-1]
'e'
```

- Slicing: Extracting a substring using the [start:stop:step] syntax

```
>>> name[2:5]
'ice'
>>> name[:]
'Alice'
>>> name[-1::-1]
'ecilA'
>>> name[0::2]
'Aie'
```

Common String Method

Method	Purpose	Example Output

.upper()	Convert all character to upper case.	>>> msg = 'Hello!'
		>>> msg
		'Hello!'
		>>> umsg = msg.upper()
		>>> umsg
		'HELLO!'

.lower()	Convert all character to lower case.	>>> lmsg = msg.lower()
		>>> lmsg
		'hello!'

.lstrip()
.rstrip()
.strip()
	These methods removes leading and trailing blank spaces from the string.	>>> msg = ' Hello! '
		>>> s1 = msg.lstrip()
		>>> s1
		'Hello! '
		>>> s2 = msg.rstrip()
		>>> s2
		' Hello!'
		>>> s3 = msg.strip()
		>>> s3
		'Hello!'

len(str)	Returns number of characters	>>> s3
		'Hello!'
		>>> len(s3)
		6
		>>> msg
		' Hello! '
		>>> len(msg)
		13

.split(<delim>)	Splits a string into a list of substrings.	>>> msg = 'This is a code to sp\lit the string'
		>>> msg.split()
		['This', 'is', 'a', 'code', 'to', 'split', 'the', 'string']
		
		>>> msg = 'This is his fish'
		>>> msg.split('is')
		['Th', ' ', ' h', ' f',

		'h']

.find(substring, start, end)
 	
'start' and 'end' is optional. And, value must be of int type.
	Returns the lowest index where substring is found. The *find()* method in Python is used to locate the first occurrence of a specified substring within a string. If the substring is found, it returns the index of its first occurrence; otherwise, it returns -1.	>>> msg
		'This is his fish'
		>>> msg.find('is')
		2
		>>> msg.find('y')
		-1
		
		>>> msg = 'This is his fish'
		>>> msg.split('is')
		['Th', ' ', ' h', ' f', 'h']
		>>> msg
		'This is his fish'
		>>> msg.find('is')
		2
		>>> msg.find('y')
		-1
		>>> msg.find('is',5)
		5
		>>> msg.find('is',6)
		9